COMPUTER VISION

LAB SHEET :05

```
# Name: Deepti
# CUID:24250168
# Roll No.: 16   Sec:B

import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure

# LABSHEET 5 - FEATURE EXTRACTION USING SIFT AND HOG
# Name: Deepti
# Roll No.: 16

# 1. LOAD THE IMAGE

# Keep image.jpg in the same folder as this Python file
image = cv2.imread("image.jpg")

if image is None:
    print("Error: Image could not be loaded.")
    print("Make sure image.jpg is in the same folder as this Python file.")
    exit()
else:
    print("Image loaded successfully.")
```

```
    print("Image size:", image.shape)

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 2. DISPLAY ORIGINAL IMAGE

plt.figure(figsize=(6, 5))
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.show()

# 3. DISPLAY GRAYSCALE IMAGE

plt.figure(figsize=(6, 5))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()

# 4. APPLY SIFT FEATURE EXTRACTION

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and calculate descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)
```

```
print("\n========== SIFT RESULTS ==========")
print("Number of SIFT Keypoints:", len(keypoints))

if descriptors is not None:
    print("SIFT Descriptor Shape:", descriptors.shape)
else:
    print("SIFT descriptors could not be calculated.")

# 5. DRAW SIFT KEYPOINTS

sift_image = cv2.drawKeypoints(
    image,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(sift_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()

# 6. EXTRACT HOG FEATURES

# Convert grayscale image to float format
gray_float = gray.astype(np.float32) / 255.0
```

```
# Calculate HOG features
hog_features, hog_image = hog(
    gray_float,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm="L2-Hys",
    visualize=True
)

print("\n========== HOG RESULTS ==========")
print("HOG Feature Length:", len(hog_features))

# 7. ENHANCE HOG VISUALIZATION

hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, np.max(hog_image))
)

# 8. DISPLAY HOG IMAGE

plt.figure(figsize=(8, 6))
plt.imshow(hog_image_rescaled, cmap="gray")
plt.title("HOG Feature Visualization")
plt.axis("off")
plt.show()
```

```
# 9. COMPARE SIFT AND HOG

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(sift_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Keypoints")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap="gray")
plt.title("HOG Features")
plt.axis("off")

plt.tight_layout()
plt.show()

# 10. BASIC IMAGE MATCHING USING SIFT

# Create a slightly modified version of the image
image2 = cv2.resize(
    image,
    None,
    fx=0.9,
    fy=0.9
)
```

```
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

# Detect SIFT features in second image
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("\n========== IMAGE MATCHING ==========")
print("Keypoints in Image 1:", len(keypoints))
print("Keypoints in Image 2:", len(keypoints2))

# Check if descriptors are available
if descriptors is not None and descriptors2 is not None:

    # Create FLANN based matcher
    index_params = dict(
        algorithm=1,
        trees=5
    )

    search_params = dict(
        checks=50
    )

    flann = cv2.FlannBasedMatcher(
        index_params,
        search_params
    )

    matches = flann.knnMatch(
```

```
    descriptors,
    descriptors2,
    k=2
)

# Apply ratio test
good_matches = []

for match_pair in matches:
    if len(match_pair) == 2:
        m, n = match_pair

        if m.distance < 0.7 * n.distance:
            good_matches.append(m)

print("Good SIFT Matches:", len(good_matches))

# Draw matched features
match_image = cv2.drawMatches(
    image,
    keypoints,
    image2,
    keypoints2,
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
```

```
    plt.figure(figsize=(12, 6))
    plt.imshow(cv2.cvtColor(match_image, cv2.COLOR_BGR2RGB))
    plt.title("SIFT Image Matching")
    plt.axis("off")
    plt.show()

else:
    print("Image matching could not be performed.")

# 11. SAVE OUTPUT IMAGES

cv2.imwrite("sift_keypoints.jpg", sift_image)
cv2.imwrite("hog_features.jpg", (hog_image_rescaled * 255).astype(np.uint8))

print("\nOutput images saved successfully!")

# OBSERVATIONS

# SIFT:
# SIFT detects important keypoints and calculates feature descriptors.
# It is robust to changes in scale and rotation.
# It is useful for image matching, object recognition, and feature detection.

# HOG:
# HOG represents an image using the distribution of gradient orientations.
# It is useful for describing object shapes and boundaries.
# HOG is commonly used in object detection applications.
```

SIFT and HOG:

SIFT focuses on local keypoints and descriptors.

HOG focuses mainly on gradient directions and object shape information.

SIFT provides strong scale and rotation robustness, while HOG is computationally

simpler and effective for shape-based object detection.

IMAGE MATCHING:

SIFT descriptors from two similar images can be compared to identify corresponding

keypoints and determine similarities between the images.

QUESTIONS AND ANSWERS

1. What is feature extraction, and why is it important in computer vision?

Answer: Feature extraction is the process of identifying important and meaningful

information from an image.

It converts raw image data into useful features that can be used for image matching,

object recognition, tracking, classification, and detection.

Feature extraction reduces complex image information into a more useful representation.

2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).

Answer: SIFT detects important keypoints in an image and creates descriptors for those keypoints.

It identifies keypoints at different scales and assigns orientations to them.

A descriptor is then generated around each keypoint to represent its local image information.

These descriptors can be compared between images for matching and recognition.

3. What are keypoints and feature descriptors in image analysis?

Answer: Keypoints are distinctive points or regions in an image that contain important information.

Examples include corners, edges, and textured regions.

Feature descriptors are numerical representations of the local information around keypoints.

They allow keypoints to be compared between different images.

4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

Answer: HOG is a feature extraction technique that represents an image using the

distribution of gradient directions.

The image is divided into small cells and gradient orientations are calculated.

These orientations are combined into histograms and normalized using blocks.

HOG is useful for describing object shape and detecting objects.

5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

Answer: SIFT is highly robust to changes in scale and rotation and is useful for

image matching and object recognition.

HOG mainly captures shape and edge information and is commonly used for object detection.

SIFT can be more computationally complex because it detects and describes local keypoints.

HOG is generally simpler and efficient for shape-based detection tasks.

6. Why is SIFT considered invariant to scale and rotation?

Answer: SIFT detects keypoints at multiple scales using a scale-space representation.

It also assigns a dominant orientation to each keypoint.

The descriptor is constructed relative to this scale and orientation.

Therefore, the same feature can be recognized even when the image is resized or rotated.

7. Mention three real-world applications where HOG descriptors are commonly used.

Answer: HOG descriptors are commonly used in pedestrian detection, vehicle detection,

and object detection systems.

They are useful because they represent the shape and edge structure of objects.

HOG can help identify objects based on their gradient and shape patterns.

8. Why is feature extraction performed before image classification or object detection?

Answer: Feature extraction converts raw image information into meaningful numerical features.

These features provide useful information about shapes, edges, textures, and important regions.

Classification and detection algorithms can then process these features more efficiently.

Therefore, feature extraction helps improve the representation of image data.

9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

Answer: Handcrafted descriptors such as SIFT and HOG are interpretable and can work well

with limited training data.

They are also useful for specific tasks and do not always require large datasets.

However, they may not perform well on highly complex image variations.

Deep learning methods can automatically learn complex features but usually require

larger datasets and greater computational resources.

10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

Answer: Feature extraction identifies important information that can be compared between images.

In image matching, feature descriptors help identify corresponding regions.

In face recognition, features can represent important facial structures.

In object detection, features describe shapes, edges, and patterns that help identify objects.

Therefore, feature extraction provides important information for many computer vision systems.

OUTPUT

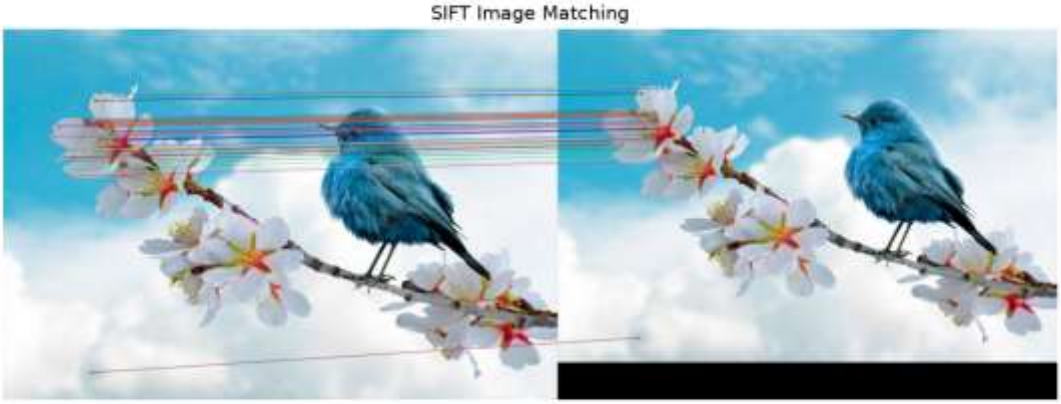

Grayscale Image


SIFT Keypoints


HOG Feature Visualization


SIFT Keypoints


HOG Features


SIFT Image Matching